



图 11.15 目录和目录项

基于目录项，NTFS 还增加了硬链接支持。每个硬链接拥有一个单独的目录项，但是其中保存的文件 ID 指向了相同的 MFT 记录。此时，该 MFT 记录中为每一个硬链接记录一个单独的文件名属性。

多数据流

文件数据是一个字符序列，又可以被称为一个数据流 (Data Stream)。NTFS 中的文件数据保存在默认的数据属性中，是文件的默认数据流。

除默认数据流之外，NTFS 还允许在同一个文件中保存多个数据流。从命名上来看，file.doc 表示该文件的默认数据流，而 file.doc:stream1 表示该文件中名为 stream1 的数据流。从存储上来看，每个数据流保存在一个单独的属性中，如默认数据流保存在默认的数据属性中，而 stream1 数据流保存在名为 stream1 的数据属性中。每个数据流有独自的一组文件属性，包括锁、流的大小等，因此多个数据流的访问之间互不干扰。多数据流允许应用程序将该文件相关的附带信息保存在该文件的特定数据流中，如图片编辑器可以将图片的缩略图保存在名为 thumbnail 的数据流中，又如编译器将程序的调试信息保存在名为 debuginfo 的数据流中。

压缩和稀疏文件

NTFS 允许对单个文件、目录中的所有文件以及整个卷 (即文件系统) 进行压缩保存。在启用压缩功能之后，存储设备上保存压缩后的数据。当应用程序访问 NTFS 上压缩保存的文件时，NTFS 会自动将被访问的那一部分文件数据解压缩，并将解压缩后的文件数据保存在内存中供给应用程序使用。该过程对应用程序来说是透明的。

除了压缩外，NTFS 还增加了对稀疏文件 (Sparse File) 的存储优化。当一个文件中存在大片连续数据为零数据 (即所有位均为 0) 时，其被称为稀疏文件。按照普通文件的方法来存储稀疏文件是很不划算的，因为其中的零数据会占用存储空间，却没有实际意义。为此，当一个文件被标记为稀疏文件时，NTFS 只保存其中的非零数据 (以及位置和长度)，其余的零数据并不保存在存储设备上。当应用程序对文件的零数据区域进行读取

时，NTFS 直接返回相应的零数据。

对于稀疏文件的优化提高了稀疏文件的存储效率和访问效率。考虑一个 10GB 的文件，其中只有开头 1GB 和结尾 1GB 中保存了有效数据，中间的 8GB 全都是零数据。如果按照普通的方法存储，则至少需要占用 10GB 存储资源，且对文件的访问需要访问复杂的文件索引找到对应的簇，再从其中读取数据。而按照优化后的方法存储，NTFS 只需要保存文件开头和结尾各 1GB 的数据，中间部分无须存储。当对中间部分进行访问时，NTFS 可以直接返回零数据，无须通过索引找到具体的簇。

11.3 虚拟文件系统

本节主要知识点

- 多个不同的文件系统如何共同工作在同一个操作系统之上？
- 为何应用程序能使用统一的接口访问多个不同文件系统上的文件？
- 伪文件系统是什么？为什么要有伪文件系统？

计算机系统中可能同时存在多种文件系统。例如，在同一台计算机中，用户分别使用 NTFS 和 Ext4 两个文件系统管理一块机械硬盘上的不同区域，用于保存电影和数据备份。同时，在一块固态硬盘中，用户使用 Ext4 文件系统保存系统文件和频繁使用的数据。当在不同的计算机系统之间传递数据时，用户还经常将 FAT32 文件系统格式的 USB 闪存盘接入计算机系统。

在操作系统中，虚拟文件系统（Virtual File System，VFS）^①对多种文件系统进行管理和协调，允许它们在同一个操作系统上共同工作。不同文件系统在存储设备上使用不同的数据结构和方法保存文件。为了让这些文件系统工作在同一个操作系统之上，VFS 定义了一系列内存数据结构，并要求底层不同的文件系统提供指定的方法，将其存储设备上的元数据统一转换为 VFS 的内存数据结构，VFS 通过这些内存数据结构向上为应用程序提供统一的文件系统服务。在本节中，我们将以 Linux 中的 VFS^[10]为例，介绍 VFS 如何管理和调用多种文件系统，以及 VFS 如何为应用程序提供统一的文件系统抽象。

11.3.1 文件系统的内存结构

在前两节中，我们主要介绍了文件系统在存储设备上的结构和操作。在实际使用的计算机中，数据需要首先被载入内存，才能被 CPU 访问和修改。因此，文件系统的正常运行离不开内存中与文件系统相关的结构。

^① 又称 Virtual File system Switch。

文件系统在使用内存时，最直接的一种方式是使用内存作为临时缓存。在每次需要访问磁盘上文件系统的结构时，文件系统分配足够多的临时内存页，再将磁盘上对应的结构读取到这些内存页中，以便直接在这些内存页上对数据进行访问。数据访问完毕之后，文件系统将释放这些临时内存页。

这种方式简单直接，但弊端也非常明显。由于每次访问均需要进行分配、访问磁盘、释放的过程，将造成比较大的性能开销。为了避免这些开销，文件系统可以延迟释放的过程，让磁盘中的结构在内存中停留更长的时间。根据数据访问的时间局部性，这些被延迟释放的结构有可能在未来再次被访问。与此同时，文件系统需要使用额外的索引结构（比如哈希表或者红黑树）管理这些延迟释放的磁盘文件系统结构，以方便在后续访问时进行查找。

对于一些常用的结构，比如超级块、分配信息和 inode 结构，文件系统会在内存中维护专有的缓存。例如，对于磁盘上的 inode 结构，在内存中具有对应的 vnode 结构。vnode 结构中保存了对应磁盘 inode 结构中的所有信息，以及用于缓存的管理和文件系统的其他信息。

除了缓存外，文件系统还需要维护一些其他结构用于文件的访问。文件系统“打开 - 读写 - 关闭”的访问模式，是一种有状态的访问方式。换句话说，文件系统需要维护一些状态表示打开的文件，这些状态将在后续读写过程中被访问和修改。举例来说，文件打开后，文件读写位置被置为 0，每次读写时，该位置会根据读写的大小而增加。因此在打开文件后，不断地调用 read 接口，最终可以将文件的全部数据读取出来。文件描述符表和文件描述表是文件系统在内存中用于管理处于打开状态的文件的主要结构。一般来说，文件描述符表和文件描述表中的内容在打开文件时创建，在用 fd 访问文件时使用，在文件关闭时删除。

图 11.16 给出了包括内存结构的文件系统结构图。应用程序通过 fd 进行文件访问时，操作系统通过 fd 找到内存中对应的文件描述结构，其中记录了对应的目标 inode、当前的文件访问位置和打开模式等。其中目标 inode 用于寻找对应的 vnode 结构。如果对应的 vnode 结构在内存中不存在，则需要分配内存空间，并将对应的 inode 结构从磁盘中读取到内存中。对于文件元数据访问，可以直接将内存 vnode 结构中的信息返回应用程序。对于文件数据访问，文件系统还需要通过内存 vnode 结构中记录的数据索引信息，找到对应的数据块位置，再通过内存中分配临时数据页，将数据从磁盘中的对应位置读取到内存中进行访问。

小思考

图 11.16 说明了基于 inode 的文件系统是如何在内存中进行缓存的。那么对于 FAT32 等磁盘中没有 inode 结构的文件系统，如何进行缓存呢？

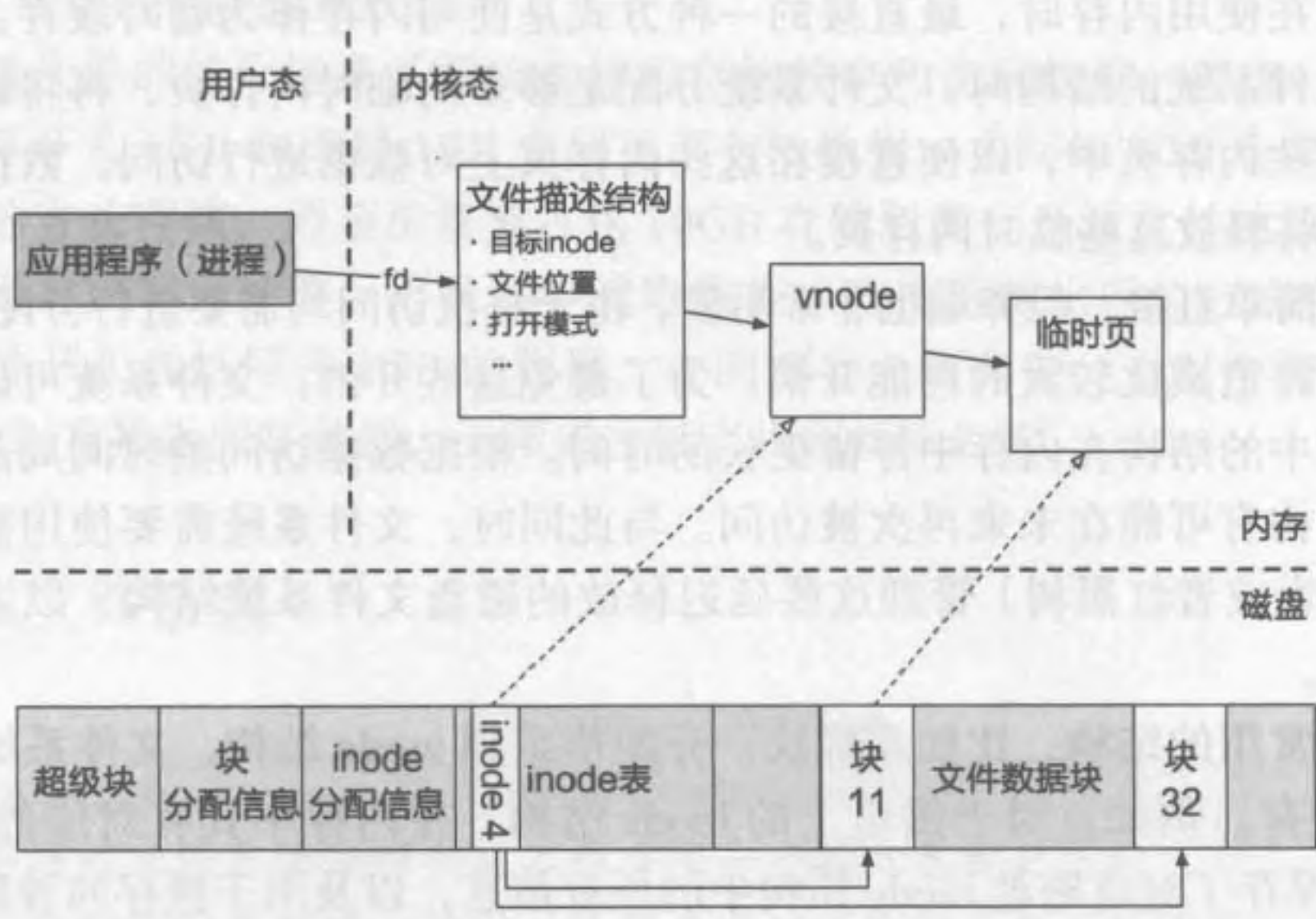


图 11.16 利用内存访问磁盘上的文件系统

文件描述符、文件描述结构和 vnode 之间的对应关系是非常灵活的。在一般情况下，打开一个文件后，一组相关联的文件描述符（fd1）、文件描述结构和 vnode 会被创建。在此情况下，若打开一个新的文件（如图 11.17a 所示），则一个新的文件描述符（fd2）被返回。该文件描述符关联了一个新的文件描述结构，以及一个与新的文件相对应的 vnode 结构。在这种情况下，两个文件的访问相互独立。若第二次打开的文件与第一个文件相同，虽然会产生新的文件描述符和文件描述结构，但是两个文件描述结构指向相同的 vnode 结构（如图 11.17b 所示）。在这种情况下，虽然通过这两个文件描述符可以访问相同的文件，但是两个文件描述符对应的文件描述结构具有不同的打开模式（比如读写、只读或者追加）和读写位置等属性。若未打开文件，而是使用 dup 系统调用从现有文件描述符（fd1）创造出一个新的文件描述符（fd2），则两个文件描述符指向相同的文件描述结构（如图 11.17c 所示）。在这种情况下，两个文件描述符的作用是完全一样的。如图 11.17d 所示，在使用 fork 等系统调用创建父子进程之后，父子进程有各自的文件描述符表，但是会共享文件描述结构，以及此后的 vnode 结构。两个进程可以通过这种方式实现进程间通信。

11.3.2 面向文件系统的接口

在本节中，我们将依次介绍 VFS 定义的内存数据结构和围绕这些数据结构而设计的文件系统方法。这些结构和方法组成了 VFS 面向文件系统的接口，使得不同文件系统可以共同在 VFS 的管理之下工作。

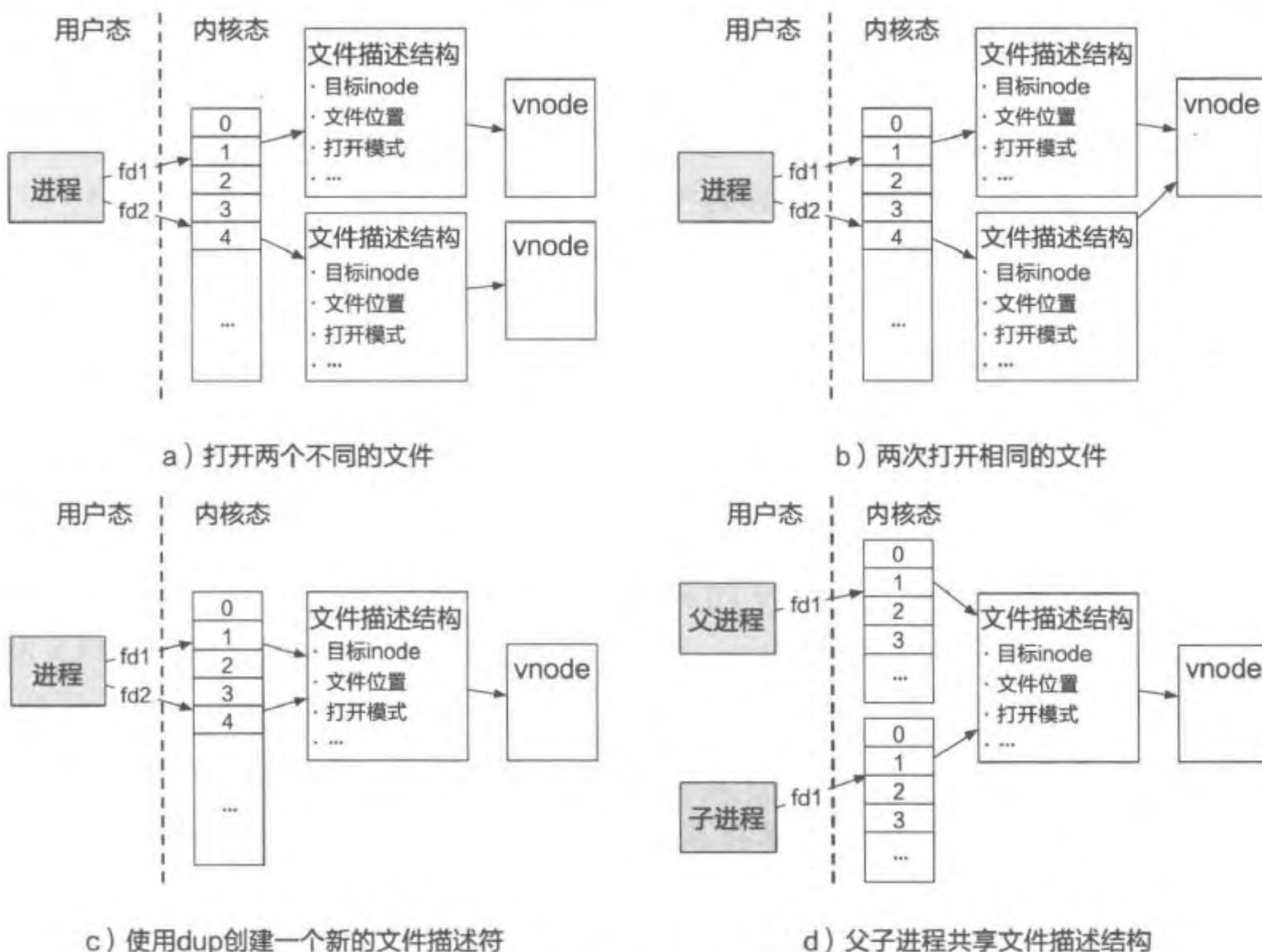


图 11.17 文件描述符表、文件描述表与 vnode 表的关系

VFS 定义的内存数据结构

Linux 的 VFS 基于 inode 制定了一系列的内存数据结构，包括超级块、inode、目录项等。这些统一的内存数据结构掩盖了多种文件系统在设计 and 实现上的不同，使得存储设备中通过不同格式存储的数据，能够以统一的格式出现在内存中。

- VFS 中的超级块。VFS 定义了自己的内存超级块结构，其中保存了文件系统的通用元数据信息，如文件系统的类型、版本、挂载点信息等。每个挂载的文件系统实例均在内存中维护一个 VFS 超级块结构。VFS 通过这些超级块结构中通用的元数据信息对多个文件系统实例进行管理。在这些通用信息之外，VFS 的超级块结构中还预留了一个指针。文件系统可以将该指针指向其特有的超级块信息。这种设计既达到了统一数据结构的目的，又保留了不同文件系统的特有信息，增加了 VFS 的灵活性。
- VFS 中的 inode。Linux 在 VFS 中同样定义了内存 inode 结构（对应前一小节中的 vnode 结构），用于保存基本的文件元数据。若文件系统想要在内存 inode 结构中记录额外信息，则需要为 VFS 的 inode 结构分配空间时多分配一些空间，之后通过计算偏移量的方式将额外信息保存在 VFS 的 inode 结构之外。为了快速定位

和使用 inode，VFS 维护了一个 inode 缓存 (icache)。VFS 的 inode 缓存使用哈希表保存了操作系统中所有的 inode 结构。当应用程序或者文件系统需要使用某个 inode，且此 inode 保存在 inode 缓存中时，可以直接从缓存中访问该 inode，从而避免了一次耗时的存储设备访问。

- VFS 中的文件数据管理。每个 VFS 的 inode 会使用基数树 (Radix Tree) 表示文件的数据。该基数树维护了从数字 i 到内存页的映射关系，内存页中记录了该文件第 i 个数据块上的数据。基数树非常适合保存文件中数据块这种以 0 开始的连续索引空间，因而被用于保存文件的页缓存。
- VFS 中的目录项。VFS 中的目录项是一个在内存中保存文件名和目标 inode 号的结构。正如 inode 缓存一样，VFS 在内存中为目录项保存了缓存，称为目录项缓存 (dcache)。

练习

11.4 当应用程序读取文件时，文件系统需要先将其保存在存储设备上的文件元数据转换成 VFS 定义的格式并保存在内存中。这是否会增加额外的开销从而影响文件系统的性能？

VFS 定义的文件系统方法

在 Linux 等宏内核中，文件系统通常作为操作系统内核的一部分对外提供服务。因此，应用程序需要通过系统调用对文件系统进行访问。图 11.18 给出了一个应用程序通过系统调用访问文件系统的例子。应用程序向操作系统内核发出读取文件的系统调用请求，操作系统内核调用 VFS 处理该请求。VFS 根据系统调用的内容和上下文，找到对应的文件系统为 Ext4 文件系统，并调用 Ext4 文件系统的“读取文件”方法，最终将请求的结果返回应用程序。

Linux 中的 VFS 以函数指针的方式定义了文件系统应提供的方法。每个 Linux 中的文件系统均需要将其方法以函数指针的方式提供给 VFS，如在文件系统代码被加载和初始化时，会告知 VFS 其文件系统类型，以及其挂载方法的函数指针。这些信息保存在代码片段 11.4 所示的结构中，由 VFS 进行组织和管理。在处理文件系统的挂载请求时，此结构中的 mount 函数便会被调用，文件系统可以进行相应的挂载操作。

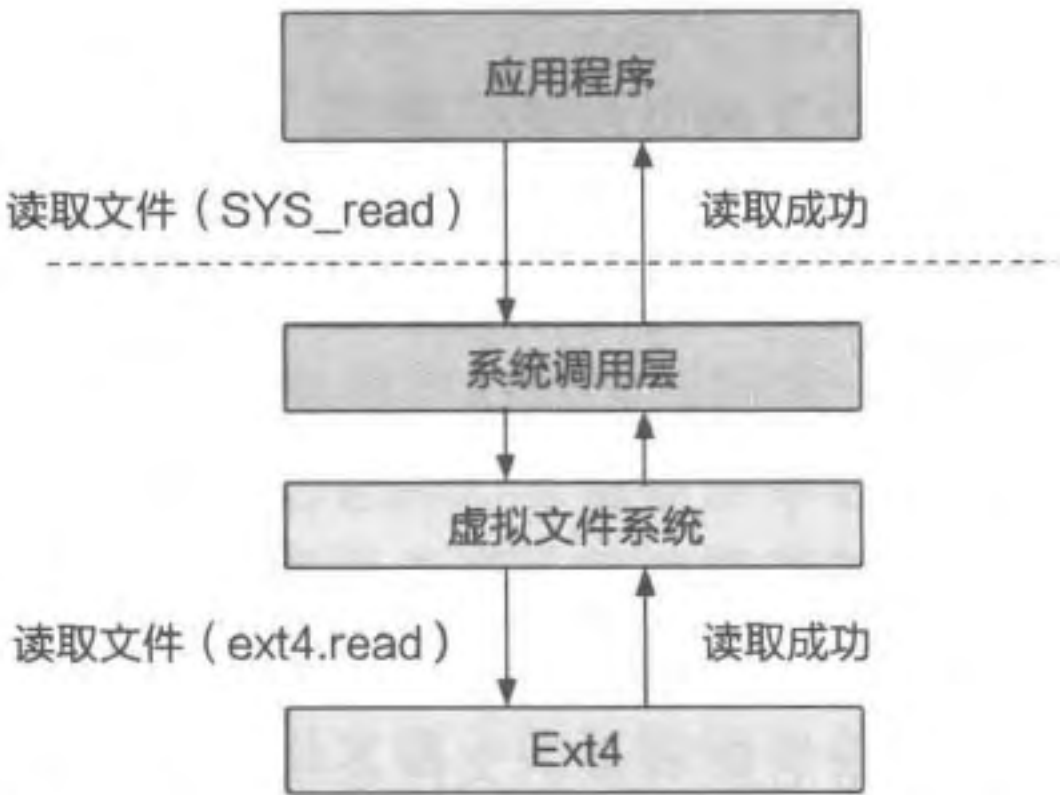


图 11.18 VFS 将应用程序与文件系统的操作进行连接

代码片段 11.4 Linux VFS 中的文件系统数据结构

```

1 struct file_system_type {
2     const char *name;
3     ...
4     struct dentry *(*mount) (struct file_system_type *,
5                             int, const char *, void *);
6     void (*kill_sb) (struct super_block *);
7     struct module *owner;
8     struct file_system_type * next;
9     struct hlist_head fs_supers;
10    ...
11 };

```

通过相似的方法，Linux 的 VFS 为其内存数据结构分别定义了不同的文件系统方法。以文件结构为例，代码片段 11.5 中列出了 Linux 的 VFS 为文件结构定义的部分方法，主要包括文件的打开（open）、读（read 和 read_iter）、写（write 和 write_iter）、定位（llseek）、内存映射（mmap）、同步写回（fsync）等操作。

代码片段 11.5 Linux VFS 在文件结构上定义的方法

```

1 struct file_operations {
2     struct module *owner;
3     loff_t (*llseek) (struct file * , loff_t, int);
4     ssize_t (*read) (struct file * , char __user * ,
5                     size_t, loff_t *);
6     ssize_t (*write) (struct file * ,
7                      const char __user * ,
8                      size_t, loff_t *);
9     ssize_t (*read_iter) (struct kiocb * ,
10                          struct iov_iter *);
11     ssize_t (*write_iter) (struct kiocb * ,
12                           struct iov_iter *);
13     long (*unlocked_ioctl) (struct file * , unsigned int ,
14                             unsigned long);
15     int (*mmap) (struct file * ,
16                 struct vm_area_struct *);
17     int (*open) (struct inode * , struct file *);
18     int (*fsync) (struct file * , loff_t, loff_t,
19                  int datasync);
20     ...
21 };

```

小知识：ioctl 接口

代码片段 11.5 中的 unlocked_ioctl 对应 ioctl 接口，其原本用于控制字符设备、网络设备和 IPC。由于 POSIX 未定义 ioctl 在其他类型文件上的行为，许多文件系统通过此接口拓展文件上的操作，为应用程序提供更多的功能。例如，文件系

统可以通过实现 `ioctl` 接口，允许应用程序设置文件的冷热程度（Hotness），以便文件系统根据数据的冷热程度使用不同的存储方法，提高存储效率。又如，为 Linux 提供硬件虚拟化支持的 KVM 模块（将在 14.7 节中进行详细介绍），也是通过实现 `ioctl` 接口提供虚拟化支持。

VFS 定义在其内存数据结构上的方法涵盖了文件系统的所有操作。当应用程序需要访问某个文件时，VFS 可以通过这些记录的函数指针，调用文件系统提供的方法，从而完成对文件系统中数据的请求。

练习

11.5 Linux 内核是宏内核，因此使用函数指针表示文件系统方法。试想一下，如果在微内核架构下，文件系统实例运行在单独的用户态进程中，能以何种方式表示文件系统方法？

VFS 与 FAT 的对接

基于 VFS 所定义的内存数据结构和方 法，Linux 可以兼容各种文件系统——不论它们是否使用 `inode` 作为主要文件系统抽象。例如，为了将基于表的 FAT32 加入 Linux 内核，需要使用一段适配代码将原有的 FAT32 文件系统实现与 VFS 中的各种数据结构和方法进行对接。

如图 11.19 所示，在打开文件时，VFS 调用 FAT32 适配代码的 `open` 函数打开文件。适配代码创建一个 VFS 的内存 `inode` 结构，通过调用原有 FAT32 的实现读取文件的元数据并将其填充到 VFS 的 `inode` 结构中。最终适配代码将 VFS 的 `inode` 结构返回 VFS，表示操作成功。

11.3.3 多文件系统的组织和管理

管理多种文件系统是 VFS 的一个重要职责。Linux 中的 VFS 通过前文中定义的一系列内存数据结构对多种文件系统进行管理。

Linux 操作系统使用简单的链表结构，将其所有支持的文件系统保存起来，包括

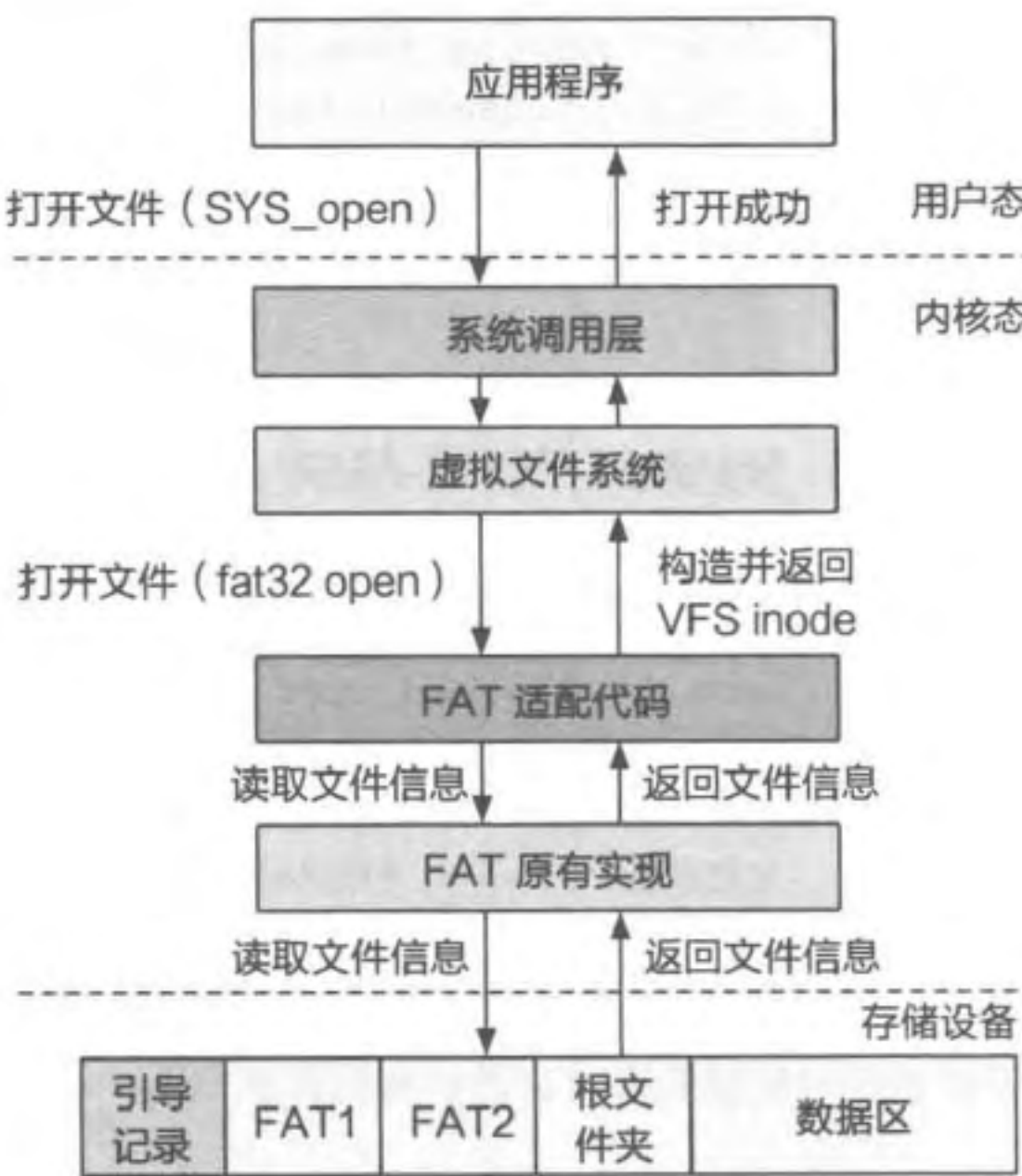


图 11.19 适配代码将 FAT32 的原有实现与 VFS 进行对接

内嵌的文件系统和在运行时作为模块加载的文件系统。在代码片段 11.4 展示的 `file_system_type` 结构中, `name` 作为文件系统类型的标识符, `next` 作为链表指针指向下一种支持的文件系统。

在进行存储设备的挂载时, VFS 通过指定的文件系统标识符, 找到对应的 `file_system_type` 结构, 并开始进行挂载操作。

挂载点

一般来说, 每个文件系统在设计时都会有其本身的内部结构。这些结构均会有一个固定的入口, 保存在超级块中。通常这个入口为文件系统的根目录。

每个文件系统都有自己的根目录, 可是应用程序所使用的路径通常只有一个根目录。当应用程序想要同时访问多个文件系统时, 如何将各个文件系统的文件系统结构结合起来呢?

一种简单的方法是将所有的文件系统放在固定的位置, 系统间相互独立。在 Windows 系统中, 我们经常会看到“C 盘”“D 盘”等, 这便是一个个挂载的文件系统。这种情况下, 新挂载的文件系统与其他文件系统在空间上形成隔离, 拥有独自的一套路径空间。

相对于 Windows 系统这种隔离开的方法, 在类 UNIX 操作系统中使用的挂载方法更加灵活。例如, Linux 的 VFS 维护一个统一的 VFS 文件系统树, 操作系统启动时有一个根文件系统, 如图 11.20 中的 FS0, 这个根文件系统作为 VFS 文件系统树的基础。

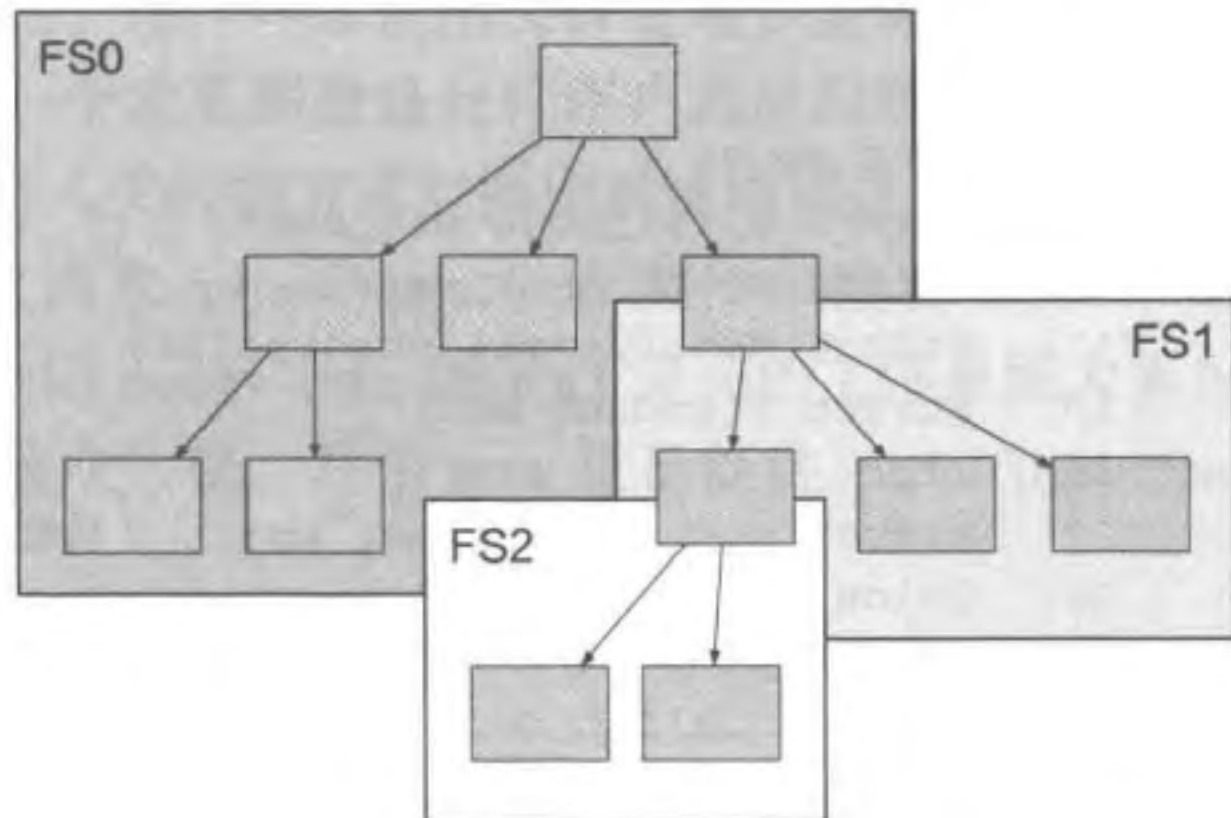


图 11.20 文件系统的挂载

在根文件系统的基础上, 其他的文件系统可以自由地被挂载到 VFS 文件系统树的任何一个目录上 (如图 11.20 中的 FS1 和 FS2)。这些作为挂载目标的目录便被称为挂载点。一旦挂载成功, 当应用程序访问挂载点及其子节点时, 便会访问到被挂载文件系统中的数据。这些挂载点就像一个个虫洞, 一旦文件访问到达挂载点, 便会跳到被挂载文件系统的根目录处继续进行访问。

小知识：在非空目录上进行挂载操作

挂载点在挂载时, 并不要求是空目录。挂载点原有的内容会被临时“覆盖”, 但并不会丢失。当所挂载的文件系统被卸载时, 这些原有内容又可以再次被访问到。

代码片段 11.6 给出了 Linux VFS 中用于保存挂载信息的部分结构。每个挂载的文件系统都有一个 `mount` 结构。其中保存了挂载点信息 (`mnt_mp`) 和挂载的文件系统信息

(mnt)。挂载点信息 (mountpoint 结构) 记录了挂载点对应的目录项等信息。挂载的文件系统信息 (vfsmount 结构) 则包含挂载文件系统超级块、挂载文件系统的根目录和挂载的选项。

代码片段 11.6 Linux VFS 中的挂载结构

```

1 struct vfsmount {
2     struct dentry *mnt_root;      // 挂载树的根目录
3     struct super_block *mnt_sb;   // 指向超级块
4     int mnt_flags;
5 };
6
7 struct mountpoint {
8     struct hlist_node m_hash;
9     struct dentry *m_dentry;      // 挂载点所在的目录项
10    struct hlist_head m_list;
11    int m_count;
12 };
13
14 struct mount {
15     struct hlist_node mnt_hash;
16     struct mount *mnt_parent;
17     struct dentry *mnt_mountpoint;
18     struct vfsmount mnt;
19     ...
20     struct mountpoint *mnt_mp;    // 挂载点
21     union {
22         // 相同挂载点上挂载的其他文件系统
23         struct hlist_node mnt_mp_list;
24         struct hlist_node mnt_umount;
25     };
26     ...
27 };

```

通过这种对挂载点的维护，Linux VFS 可以灵活地挂载多种文件系统，让多种不同的文件系统在相同的操作系统下共同工作。

小知识：不同设备上挂载相同类型的文件系统

同一类型的文件系统可以在不同设备上进行挂载。例如当 Linux 上的 /dev/sda2 和 /dev/sdb1 同时为 Ext4 文件系统格式时，它们可以同时被挂载到不同目录。虽然它们使用同一份 Ext4 文件系统的代码逻辑，但所维护和操作的数据结构实例（如超级块等）是不同的。

11.3.4 伪文件系统

文件接口是一种非常灵活的数据访问方式，不仅可以用来访问用户数据，还可以

用于其他功能。Linux 实现了一些不用于保存文件数据的文件系统，称为伪文件系统 (Pseudo File System)。表 11.4 给出了一些常见的 Linux 伪文件系统。

表 11.4 Linux 中的一些伪文件系统

名 称	常用挂载点	描 述
procfs	/proc	查看和操作与进程相关的信息和配置
sysfs	/sys	查看和操作与进程无关的系统配置
debugfs		用于内核状态的调试
cgroupfs		用于管理系统中的 cgroups
configfs	/sys/kernel/config	创建、管理和删除内核对象
hugetlbfs		查看和管理系统中的大页信息

通过使用文件的抽象，伪文件系统能够直接获得内核中 VFS 提供的命名、权限检查等功能。同时，由于文件接口的简单易用性，用户可以使用文件管理、查看、监控等工具与伪文件系统进行交互。伪文件系统的一个常见用途是允许用户态应用程序通过读取文件的方式读取内核提供的信息，并通过写入文件的方式对操作系统内核进行配置调整。例如，在输出记录 11.3 中，我们可以通过读取 /proc/filesystems 文件来查看当前 Linux 操作系统内核中支持的所有文件系统。其中以 nodev (no device) 开头的文件系统大多为伪文件系统，即并非直接从设备中保存和读取数据。其中 proc 正是提供 /proc/filesystem 这个文件接口的伪文件系统。

输出记录 11.3 使用伪文件系统中的文件查看所有支持的文件系统类型

[user@osbook ~] # cat /proc/filesystems
nodev sysfs
nodev tmpfs
nodev bdev
nodev proc
nodev cgroup
nodev cpuset
nodev devtmpfs
nodev configfs
nodev debugfs
nodev tracefs
nodev securityfs
nodev sockfs
nodev bpf
nodev hugetlbfs
nodev devpts
nodev autofs
nodev pstore
ext3
ext2
ext4

伪文件系统同样使用 VFS 定义的内存数据结构和方法。当伪文件系统中的文件被读取时，VFS 在进行路径解析后，会发现目标文件被该伪文件系统管理。因此，VFS 会通过记录内存数据结构中的文件系统方法，将文件请求交由该伪文件系统进行处理。之后，伪文件系统便可以根据文件名等特征进行特定的内核操作。如上述 `/proc/filesystems` 文件被读取时，`/proc` 伪文件系统会遍历保存在内存中的所有文件系统类型（`file_system_type` 结构），并将其逐一写入用户态传递的缓冲区中。同理，当伪文件系统中的文件被写入时，伪文件系统会根据请求类型对内核状态进行改变。

伪文件系统提供了一种在系统调用之外，应用程序和操作系统内核进行交互的方法，提高了操作系统的灵活性。同时由于用户态有大量的文件操作工具（如 `cat`、`echo`、重定向等）和脚本，系统管理员可以使用自己熟悉的工具，轻松地利用伪文件系统的文件接口检查和管理操作系统内核状态。

11.4 VFS 与缓存

在前面的讨论中，我们假设对文件系统中结构的修改都是直接在存储设备中进行的。然而，在存储设备上直接访问数据存在两个问题：首先，目前大多数存储设备的读写粒度为一个块（常见为 512B 或 4KB），而文件系统所进行的更改往往并非对齐到块的边界，其读写的字节数也并非恰好为块大小的整数倍；其次，存储设备的访问速度相比内存慢几个数量级，大量频繁的存储设备访问操作会成为应用程序的性能瓶颈。

11.4.1 访问粒度不一致问题和一些优化

文件系统使用内存作为中转来解决软件和硬件的访问粒度不一致问题。为了便于描述，我们假设一个存储设备的块大小为 4KB。当文件系统需要修改存储设备中的 8 字节时，需要先从存储设备中找到这 8 字节所在的块号，并将整个块的数据读入一个 4KB 的内存页中。文件系统在内存页中修改这 8 字节，并将修改后的内存页通过驱动写回存储设备中。这是一个“读取 - 修改 - 写回”的过程。

之所以要在修改和写回之前先读取，是因为我们只想修改存储块中的一部分，而写回操作会覆盖整个存储块。为了保证此存储块中其他部分的数据不变，需要先将这些数据读出来，之后随修改后的数据一同写回存储设备。一个明显的问题是：若每个文件请求中每个结构的修改都经过完整的“读取 - 修改 - 写回”，将会产生多次的存储设备访问。一些比较简单的优化可以避免不必要的存储设备访问。如果一次修改的数据量刚好覆盖了整个存储块，那么可以不用读取，直接将修改后的 4KB 数据写回存储设备中。此外，如果一个文件请求中的多次修改均是在同一个存储块中，那么可以将多次修改合并到一个“读取 - 修改 - 写回”过程中，即变为“读取 - 修改 1 - 修改 2 - 修改 3 - ... - 修改 n - 写回”。这样同样是访问了两次存储设备，却可以完成多次修改。